

# Beyond Branch Prediction: A Monte Carlo Validation of Semi-Static Dispatch in C++17 for Ultra-Low Latency Systems

Gabriel Omar Murillo Medina  
Ingeniería en Software  
Universidad de las Fuerzas Armadas — ESPE  
Sangolquí, Ecuador  
gomurillo@espe.edu.ec

**Resumen**—En sistemas de trading de alta frecuencia (HFT), las penalizaciones por predicción fallida de ramas (*branch misprediction*) representan una fuente crítica de latencia no determinista. Este trabajo implementa y valida experimentalmente la técnica de condiciones semi-estáticas propuesta por Bilokon et al. (2025) mediante `FastBranch`, una abstracción *header-only* en C++17 que desacopla la evaluación de condiciones del flujo de ejecución crítico. Se condujo un análisis Monte Carlo con  $K = 100$  ensayos independientes ( $N = 5 \times 10^6$  operaciones cada uno), totalizando  $5 \times 10^8$  observaciones. Los resultados demuestran un *speedup* estadísticamente significativo de  $3.11 \times$  ( $t = 33.28$ ,  $p < 0.001$ , Cohen's  $d = 4.71$ ), reduciendo la latencia de 13.45 ns/op a 4.39 ns/op — convergiendo al límite teórico del procesador (4.10 ns/op). Se identifican además condiciones de contorno donde la técnica resulta contraproducente, incluyendo un *overhead* del 20.2% por *type erasure* en `std::function` y degradación por contención de caché en escenarios multi-hilo.

**Index Terms**—branch misprediction, semi-static conditions, function pointer dispatch, speculative execution, pipeline optimization, C++17 metaprogramming, low-latency software architecture, Monte Carlo simulation, statistical validation, high-frequency trading



## 1. INTRODUCCIÓN

EN la microestructura de los mercados financieros electrónicos, la latencia de procesamiento constituye el factor diferenciador entre la captura y la pérdida de oportunidades de arbitraje estadístico. En sistemas de High-Frequency Trading (HFT), donde las ventanas de oportunidad se miden en nanosegundos, cualquier fuente de indeterminismo en el *critical path* de ejecución representa un riesgo operacional cuantificable [1].

Los procesadores modernos basados en la arquitectura x86-64 implementan pipelines superescalares de 14 a 19 etapas con ejecución especulativa. Ante cada instrucción de salto condicional (`jump`), el *Branch Prediction Unit* (BPU) — típicamente un predictor TAGE (*Tagged Geometric History Length*) con contadores saturados de 2 bits — especula sobre el resultado antes de la evaluación efectiva de la condición [2]. Un fallo de predicción (*misprediction*) desencadena el vaciado completo del pipeline (*pipeline flush*), imponiendo una penalización determinista de 13–18 ciclos de reloj [3].

Bilokon et al. [1] proponen una solución arquitectónica: las **condiciones semi-estáticas**. La técnica consiste en eliminar completamente la instrucción condicional del *hot path* mediante Self-Modifying Code (SMC), delegando la actualización del estado a una *cold path* asíncrono.

El presente trabajo persigue cuatro objetivos:

1. Cuantificar el *speedup* real de la técnica semi-estática sobre `switch/if-else` con condiciones uniformemente

aleatorias.

2. Medir el *overhead* de `std::function` frente a punteros crudos a función.
3. Evaluar la viabilidad del multi-threading para este patrón.
4. Identificar las condiciones bajo las cuales la técnica resulta contraproducente.

## 2. MATERIALES Y MÉTODOS

### 2.1. Diseño Experimental

Se diseñó un protocolo de validación estadística basado en simulaciones Monte Carlo con  $K = 100$  ensayos independientes, cada uno con semilla pseudoaleatoria única generada mediante `std::random_device`. Cada ensayo procesó  $N = 5 \times 10^6$  operaciones de *dispatch* condicional, totalizando  $5 \times 10^8$  observaciones experimentales.

Se evaluaron cuatro estrategias de *dispatch*:

- **switch nativo:** Evaluación condicional en cada iteración (línea base).
- **FastBranch:** Propuesta experimental con punteros crudos y actualización infrecuente ( $f_{\text{set}} = 1/1000$ ).
- **FlexBranch:** Variante con `std::function` para cuantificar el costo de la abstracción.
- **Control directo:** Llamada sin *dispatch* condicional (límite teórico).

Las diferencias se validaron mediante pruebas  $t$  de Welch (bilateral,  $\alpha = 0,05$ ). Se reporta el tamaño del efecto mediante la  $d$  de Cohen.

## 2.2. El Problema: Pipeline Flush

La Fig. 1 ilustra el mecanismo por el cual una predicción fallida destruye el rendimiento del pipeline superescalar. Cuando el BPU falla, todas las instrucciones ejecutadas especulativamente se descartan, generando una burbuja de 14–18 ciclos.

## 2.3. Implementación y Arquitectura

Se desarrolló la biblioteca `semi_static.hpp` (header-only, C++17) con cuatro variantes (Cuadro 1). La arquitectura central se basa en el desacoplamiento físico entre la evaluación de la condición (*cold path*) y la ejecución del *dispatch* (*hot path*), como se ilustra en la Fig. 2.

Cuadro 1  
Variantes implementadas en `semi_static.hpp`

| Clase            | Mecanismo        | Heap | Thread-safe |
|------------------|------------------|------|-------------|
| FastBranch       | Array FuncPtr    | No   | No          |
| AtomicFastBranch | Ídem + atomic    | No   | Sí          |
| FlexBranch       | vector<function> | Sí   | No          |
| AtomicFlexBranch | Ídem + atomic    | Sí   | Sí          |

## 2.4. Entorno de Ejecución

Las pruebas se ejecutaron en Windows 11, procesador Intel de 12 núcleos lógicos, compilador GCC 11.2 (MinGW-w64, C++17, -O2). La instrumentación temporal se realizó con `std::chrono::high_resolution_clock`. Las condiciones aleatorias se pre-generaron antes de cada *trial* para excluir el costo del RNG.

## 3. RESULTADOS

### 3.1. Análisis Monte Carlo: Reducción de Latencia

El Cuadro 2 resume las estadísticas descriptivas obtenidas sobre los 100 ensayos independientes. La Fig. 3 visualiza la comparativa.

Cuadro 2  
Estadísticas descriptivas de latencia ( $K = 100$ ,  $N = 5 \times 10^6$ )

| Estrategia        | $\bar{x}$ (ns/op) | $\sigma$ (ns) | IC 95 %             | Speedup      |
|-------------------|-------------------|---------------|---------------------|--------------|
| switch            | 13,45             | 2,58          | [12,92, 13,97]      | 1,00×        |
| FlexBranch        | 5,28              | 1,90          | [4,89, 5,67]        | 2,55×        |
| <b>FastBranch</b> | <b>4,39</b>       | <b>0,87</b>   | <b>[4,21, 4,57]</b> | <b>3,11×</b> |
| Control directo   | 4,10              | 1,01          | [3,89, 4,31]        | 3,28×        |

### 3.2. Validación Estadística

La prueba  $t$  de Welch (bilateral,  $\alpha = 0,05$ ) confirmó la significancia de todas las comparaciones (Cuadro 3).

El IC al 95% del *speedup* de FastBranch fue  $[3,00 \times, 3,23 \times]$ : en ninguno de los 100 ensayos bajó de  $3 \times$ .

Cuadro 3  
Resultados de la prueba  $t$  de Welch ( $\alpha = 0,05$ , bilateral)

| Comparación              | $t$   | $d$  | $p$     | Veredicto |
|--------------------------|-------|------|---------|-----------|
| switch vs FastBranch     | 33,28 | 4,71 | < 0,001 | Masivo    |
| switch vs FlexBranch     | 25,51 | 3,61 | < 0,001 | Grande    |
| FastBranch vs FlexBranch | -4,24 | 0,60 | < 0,001 | Medio     |
| FastBranch vs Control    | 2,19  | 0,31 | 0,031   | Pequeño   |

## 3.3. Hallazgos Negativos

### 3.3.1. Contención de Caché en Multi-threading

Al evaluar AtomicFastBranch con 11 *hot threads* compartiendo contadores atómicos, el rendimiento colapsó a  $0,43 \times$  del caso mono-hilo. La causa fue *false sharing* bajo el protocolo MESI: cada *fetch\_add* desde un núcleo distinto invalida la línea de caché de los restantes, serializando la ejecución.

### 3.3.2. Predictor vs. Semi-Static

Cuando la condición cambia infrecuentemente ( $1/10^4$  operaciones), el BPU acierta >99.9% de las veces. En este escenario, *if/else* superó a FastBranch por un 26%, dado que el costo del *call* indirecto excede al del salto correctamente predicho.

## 4. DISCUSIÓN

Los resultados son consistentes con Bilokon et al. [1], quienes reportaron  $\sim 58\%$  de reducción temporal. Nuestro *speedup* de  $3,11 \times$  (68% de reducción) es superior, atribuible a funciones rama deliberadamente ligeras que aíslan el costo puro del *dispatch*.

La convergencia de FastBranch al límite teórico (brecha de  $0.29 \text{ ns/op} \approx 1 \text{ ciclo @ } 3 \text{ GHz}$ ) corresponde exactamente al costo de una lectura L1 + *call* indirecto, confirmando la eliminación total de la penalización por *misprediction*.

La disparidad en la varianza ( $\sigma_{\text{Fast}} = 0,87$  vs.  $\sigma_{\text{switch}} = 2,58$ ) constituye un resultado igualmente significativo: FastBranch reduce el *jitter* en un 66%, aportando determinismo temporal crítico para HFT.

**Guía de aplicabilidad:** La técnica es recomendable exclusivamente cuando (a) la condición es impredecible para el BPU, (b) las funciones rama son computacionalmente ligeras ( $< 100 \text{ ns}$ ), y (c) la actualización del estado ocurre al menos  $10^3$  veces menos frecuente que la ejecución.

## 5. CONCLUSIONES

- FastBranch alcanza  $4.39 \text{ ns/op}$ , convergiendo al límite teórico ( $4.10 \text{ ns/op}$ ) con un *speedup* de  $3,11 \times$ , validado estadísticamente ( $d = 4,71$ , masivo).
- La varianza inter-trial ( $\sigma = 0,87 \text{ ns}$ ) es 66% inferior a la de switch, aportando determinismo temporal.
- `std::function` introduce 20.2% de *overhead* por *type erasure*, confirmando la necesidad de punteros crudos en el *hot path*.
- La técnica es contraproducente cuando el BPU predice correctamente el patrón (>99.9% de acierto con condiciones estables).

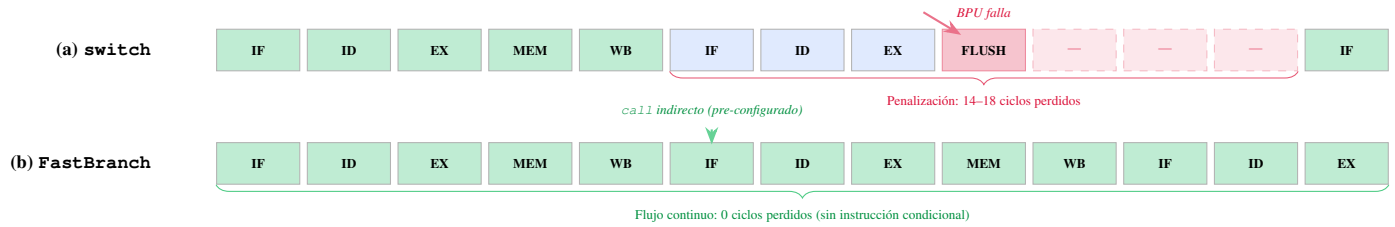


Figura 1. Comparativa del pipeline del CPU. (a) Con *switch*: una predicción fallida del BPU provoca el vaciado del pipeline (*flush*), desperdiciando 14–18 ciclos. (b) Con *FastBranch*: no existe instrucción condicional; el salto indirecto está pre-configurado y el pipeline fluye sin interrupción.

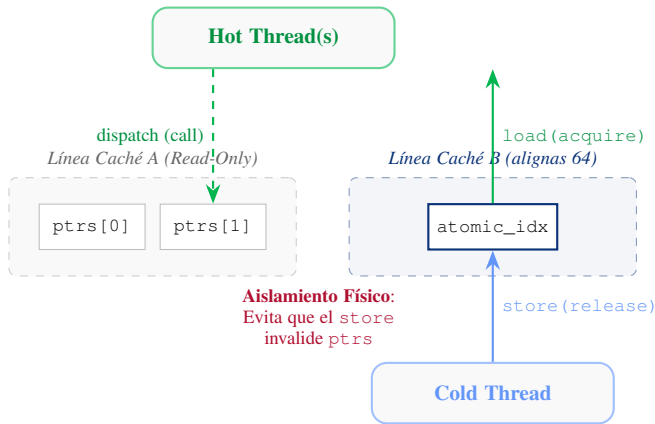


Figura 2. Arquitectura de memoria y sincronización inter-hilo para HFT. La directiva *alignas(64)* aísla el índice atómico en una línea de caché exclusiva. Esto previene que las actualizaciones del *Cold Thread* invaliden el caché L1 de los punteros a función (*false sharing*). La semántica *release/acquire* garantiza visibilidad de memoria sin utilizar costosos bloqueos.

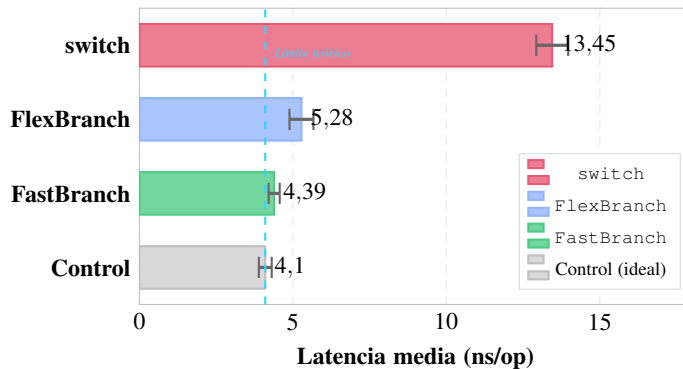


Figura 3. Latencia media por estrategia de *dispatch* con intervalos de confianza al 95%. La línea punteada indica el límite teórico del procesador. *FastBranch* converge a este límite.

## REFERENCIAS

- [1] P. A. Bilokon, M. Lucuta, and E. Shermer, “Semi-static conditions in low-latency C++ for high frequency trading: Better than branch prediction hints,” *Journal of Parallel and Distributed Computing*, vol. 196, p. 105000, Feb. 2025. doi: [10.1016/j.jpdc.2024.105000](https://doi.org/10.1016/j.jpdc.2024.105000).
- [2] Intel Corporation, *Intel® 64 and IA-32 Architectures Optimization Reference Manual*, Order No. 248966-045, Jun. 2023. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [3] A. Fog, “The microarchitecture of Intel, AMD, and VIA CPUs: An optimization guide for assembly programmers and compiler makers,” Copenhagen University College of Engineering, Tech. Rep., 2023. [Online]. Available: <https://www.agner.org/optimize/microarchitecture.pdf>

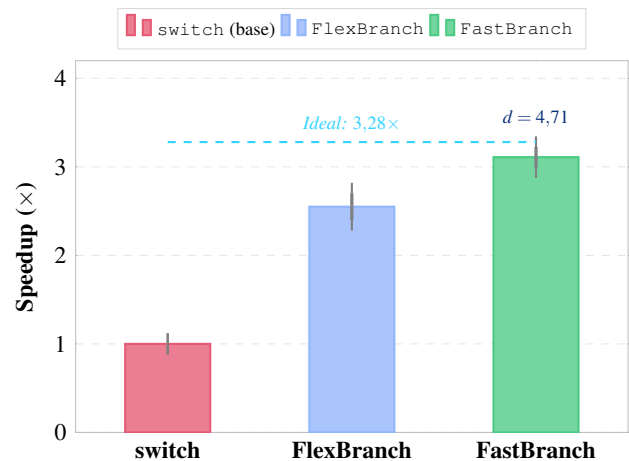


Figura 4. *Speedup* relativo al *switch* nativo con IC al 95%. Cohen's  $d = 4.71$  indica diferencia masiva. La línea punteada muestra el límite sin *dispatch* condicional.